

fwforge

Palo Alto → FortiGate Conversion

Capability overview and comparison with FortiConverter

An open, clean-room PAN-OS → FortiOS converter. Prepared for Fortinet engineering review · June 2026 · fwforge v0.54.0

Contents

1. Executive summary

fwforge is a transparent, fully local tool that converts Palo Alto PAN-OS configurations — firewall XML, set/display-set format, and Panorama exports — into clean, restorable FortiOS configuration. It exists to make migrating off Palo Alto and onto FortiGate fast, auditable, and trustworthy.

- **Clean-room.** It contains no Fortinet or FortiConverter code, and no licensed FortiGuard data files. It targets the public FortiOS CLI only.
- **An on-ramp to FortiGate.** Every conversion is a Palo Alto estate moving onto FortiOS. The tool's purpose is to remove migration friction from that path.
- **Full security-profile coverage.** App-ID (per-application signatures), URL filtering (FortiGuard categories + explicit URL lists), file blocking, antivirus, WildFire (→ FortiSandbox), and IPS (anti-spyware / vulnerability) all convert — every PAN security-profile type except Data Filtering — each schema-certified against the live target.
- **Parity on the core, ahead on assurance.** It matches FortiConverter on objects / policy / NAT / routing / VPN, and now on App-ID granularity, and goes beyond it on output quality: every emitted config is schema-certified against the exact target firmware build, is deterministic and git-diffable, and carries per-line provenance back to the source line.

The conversions and signature data in this document were **verified live against a FortiGate-601F running FortiOS 8.0.0 build0167** — the FortiGuard application-signature database, web categories, file types, and IPS filters were read from the device, and the emitted App-ID / web-filter / file-filter / antivirus / IPS / FortiSandbox output was schema-certified clean (0 unknown tables, 0 unknown attributes).

2. What makes it different

These properties distinguish fwforge from a typical converter. They are the reasons the output can be trusted on a production cutover.

- **Mappings sourced from your own FortiGate.** Per-application App-ID, FortiGuard web categories, file types and IPS filters come from the target device's own FortiGuard database — read live (one read-only API call), cached locally, refreshable on demand — not a bundled third-party table that drifts. A staleness warning fires when the cache ages.

- **Schema-certified output.** Every section and attribute is validated against the exact target firmware's CLI schema, fetched live from the device (one read-only API call) or from cache. Unknown table = error; unknown attribute = warning. Nothing reaches a device unverified.
- **Deterministic and diffable.** No timestamps, no random ordering. Re-running the same input yields byte-identical output, so a reviewer can diff two conversions in git and see exactly what changed.
- **Per-line provenance and a real report.** Every converted object records its source file and line. Findings are a structured report (Markdown + JSON + HTML) with severity and location — not warnings buried as comments in the config. A non-zero exit code is returned on errors.
- **Version-delta awareness.** fwforge compares source and target FortiOS versions and reports removed or introduced features, auto-fixed attribute/section renames, and silent default-flips — in both upgrade and downgrade directions.
- **Never silently broadens or drops.** Anything it cannot faithfully convert is flagged, never approximated into something more permissive. For example, a custom udp/53 service is not collapsed onto the FortiOS built-in DNS service, which is tcp+udp/53.
- **Local and target-model aware.** It runs as a single local package with no cloud dependency, and maps interfaces against the destination model's real port inventory (read from a backup or model table) using a FortiOS-style faceplate.

3. PAN-OS → FortiGate capability matrix

Everything fwforge converts from a Palo Alto source. Anything outside this table is reported as a finding, not dropped silently.

Domain	What fwforge converts	Notes
Input formats	PAN-OS firewall XML and set / display-set format; Panorama exports	One unified parser, auto-detected. Panorama template-merged running-configs handled.
Device system	deviceconfig hostname, DNS (dns-setting), and NTP (ntp-servers) → config system global / dns / ntp	Emitted ahead of the security config; classifies into config global under a VDOM wrap, once per device for multi-vsys.
Panorama & multi-vsys	Device-group selection; shared + pre/post rulebases merged in PAN evaluation order; optional template for network config; each vsys → its own FortiOS VDOM — interfaces hoisted to config global with set vdom, a management VDOM designated	Cross-vsyes references flagged.
Interfaces	Physical, aggregate (LAG), Layer-3 subinterfaces (VLAN), loopback, tunnel	Mapped to the destination model's real ports via a faceplate UI with positional auto-guess; promote-a-port-to-LAG and “do not map” supported. Created VLAN / aggregate / loopback names are clamped to FortiOS's 15-char interface-name limit, references remapped.
Aggregates / LAG	PAN aggregate-ethernet → FortiOS 802.3ad aggregate (type aggregate + member ports + lacp-mode)	LACP mode read from source; emitted before the VLANs that ride it so the script loads in order.
Zones	PAN security zones → FortiOS zones	Emitted with intrazone allow to preserve PAN's default same-zone behavior; flagged for tightening.
Addresses	host / subnet / range / FQDN; IPv4 and IPv6 (address6)	Names sanitized to FortiOS limits with every reference remapped.
Address groups	Nested groups with single-family enforcement	Mixed-family members handled per FortiOS rules.
Services	Custom TCP/UDP (port + source-port ranges), service groups, predefined service-http/https	Mapped to FortiOS built-ins only on exact semantic match — never broadened.

App-ID	App-IDs → application-control with PER-APPLICATION signatures (matched to the FortiGuard app DB), category fallback for the rest; App-IDs → policy service (standard ports); application-groups flattened to leaves	Signature-level when an app DB is present (verified live, ~3,300 sigs). App-IDs matching a FortiOS built-in service emit that named service.
URL filtering	PAN url-filtering → FortiOS webfilter: FortiGuard categories + custom URL lists (custom-url-category) → a webfilter urlfilter table for per-URL allow/block	Category IDs verified live. PAN action → ftgd-wf action; URL with * → wildcard, else simple.
File blocking	PAN file-blocking profiles → FortiOS file-filter profiles	File types verified live. PAN action → block / log-only / warning.
Antivirus	PAN antivirus/virus → FortiOS antivirus profile (per-protocol av-scan block / monitor / disable from the PAN decoders)	FortiGuard AV engine + signatures do the scanning; per-protocol scan intent carried.
WildFire	PAN wildfire-analysis → FortiSandbox submission folded into the antivirus profile (analytics-db + per-protocol fortisandbox)	Needs a device-level FortiSandbox appliance / Cloud (config system fortisandbox); flagged.
IPS	PAN anti-spyware + vulnerability → one FortiOS IPS sensor: severity-filter entries + exact CVE-filter entries, FortiGuard-recommended baseline (action default)	Posture parity, not signature-for-signature — CVE is the exact cross-vendor key; per-threat exceptions flagged (see §6).
SSL inspection	webfilter / file-filter / AV / IPS policies attached with the built-in certificate-inspection profile	SNI-based; no CA rollout. Switch to deep-inspection for full HTTPS content control.
Security policies	PAN rules → FortiOS firewall policy: zones, addresses, services, action, logging, disable state, source/destination negation	Per-rule selection (exclude rules); schedules flagged; application-default handled.
NAT	Source NAT (interface PAT) → policy NAT or central-snat-map; destination/static NAT → VIP (+ port-forward); subnet 1:1 DNAT → range VIP	NAT mode selectable: policy or central.
Routing	Static routes (egress inferred when omitted); BGP (AS, router-id, neighbors, networks, redistribute); OSPF (areas, networks, passive); IPv6 routes	The route table also drives policy dstintf inference (longest-prefix-match).
IPsec VPN	Site-to-site → route-based phase1/phase2-interface + tunnel routes + bidirectional	Encrypted / exported PSKs detected → placeholder + error (never a silently broken

	policies; IKE crypto, proposals, PFS, IKEv2	secret).
Output hygiene	Prune unreferenced objects (iterative); merge duplicate objects; split interface pairs; include/exclude rules	Acts on the output, not just reports.
Output packaging	config-all.txt + per-branch script files; findings embedded as # comments	Deterministic ordering; restore-safe.
FortiManager	Optional JSON-RPC import bundle: object creates + a policy package for an ADOM	
Assurance	Schema certification; version-delta scan; Markdown / JSON / HTML report; XML coverage map (% of source read)	See §2 and §5.

4. Security-profile conversion

Every PAN security-profile type converts except Data Filtering. A rule's profile-setting (direct refs or a profile-group) resolves to the matching FortiOS UTM profiles, built lazily and deduplicated, and attached to the policy.

Converted

- **App-ID → Application Control (per-application).** A rule's App-IDs map to specific FortiOS application signatures via the target's FortiGuard app DB (e.g. Facebook, Microsoft.Teams, Microsoft.365), with FortiGuard categories as the fallback for unmatched apps; in parallel the apps' standard ports fill the policy service. Application-groups expand to their leaf apps.
- **URL filtering → Web Filter.** PAN url-filtering profiles become FortiGuard-category webfilter profiles (one PAN category can expand to several, e.g. alcohol-and-tobacco → Alcohol + Tobacco), and PAN custom-url-category URL lists become a FortiOS webfilter urlfilter table — per-URL allow/block (wildcard or simple) carried over.
- **File blocking → File Filter.** PAN file-blocking → FortiOS file-filter; PAN file types map to FortiOS file types over HTTP/FTP/SMTP/IMAP/POP3/MAPI/CIFS/SSH.
- **Antivirus → Antivirus profile.** PAN virus decoders map to per-protocol av-scan (block / monitor / disable). The FortiGuard engine and signatures do the scanning; the per-protocol scan intent is carried.
- **WildFire → FortiSandbox.** A wildfire-analysis reference folds FortiSandbox submission into the rule's antivirus profile (analytics-db + per-protocol fortisandbox). Requires a FortiSandbox appliance or FortiSandbox Cloud (device-level), which the report flags.
- **Anti-Spyware + Vulnerability → IPS sensor.** Merged into one FortiOS IPS sensor: severity rules become severity-filter entries (FortiGuard-recommended action as the baseline), and CVE-pinned rules become exact CVE-filter entries — the one real cross-vendor key (e.g. Log4Shell maps precisely). This

is posture parity, not signature-for-signature; per-threat exceptions and DNS sinkhole are flagged (see §6).

- **SSL inspection.** Policies carrying a web / file / AV / IPS profile attach the built-in certificate-inspection profile (SNI-based, no certificate rollout). Deep-inspection is recommended in the report for full HTTPS content control.

Not converted

Data Filtering (→ FortiOS DLP) is the only remaining PAN profile type; it is flagged per rule for manual configuration. PAN risk-level URL buckets (high/medium/low-risk) and any unmapped category, file type, or App-ID are likewise flagged — never silently dropped.

5. Output quality and assurance

- **Schema certification.** The emitted CLI is parsed and checked table-by-table, attribute-by-attribute, against the target build's schema. In this review a full PAN→FortiGate conversion certified clean against a live 601F (FortiOS 8.0.0 build0167): 0 unknown tables, 0 unknown attributes.
- **Deterministic output.** No timestamps or nondeterministic ordering, so successive runs diff cleanly.
- **Structured findings.** Markdown, JSON, and HTML reports, each finding tagged with severity, area, and source file:line. The HTML report is print-to-PDF friendly for hand-off.
- **Coverage map.** The parser reports the percentage of source values it read and flags any unread subtree, so reviewers can see what was and was not considered.

6. Scope and limitations (stated plainly)

Engineers should know exactly where the tool stops. Each of these is surfaced in the report; nothing fails silently.

- **IPS is posture parity, not signature parity.** There is no Palo Alto Threat-ID → FortiGuard-signature crosswalk — for any tool — so PAN anti-spyware / vulnerability profiles map at the severity + CVE level (with FortiGuard's recommended action as the baseline), not signature-for-signature. CVE-pinned rules map exactly; per-threat exceptions and DNS sinkhole are flagged for manual review. Validate IPS before enforcing.
- **App-ID DB freshness.** Per-application App-ID uses the target FortiGate's FortiGuard signature DB, cached locally; conversions are deterministic and offline and do not refetch per run. FortiGuard adds App-IDs continuously, so the tool warns when the cache is stale — refresh with one read-only call.
- **Data Filtering → DLP.** The only PAN security-profile type not yet converted; flagged per rule. FortiOS ships the building blocks (DLP profiles + def-cc / def-ssn dictionaries) so this is a planned addition.
- **FortiSandbox is a device dependency.** WildFire conversion enables FortiSandbox submission in the AV profile, but a FortiSandbox appliance or Cloud must be configured device-level (config system fortisandbox) — outside the converted policy package.

- **Complex NAT.** Common source and destination NAT convert; exotic twice-NAT idioms are flagged for manual handling.
- **Remote-access VPN.** Site-to-site IPsec converts; GlobalProtect / remote-access is not converted.

7. fwforge vs FortiConverter

A feature-and-quality comparison for the Palo Alto → FortiGate path. Both tools convert the core object/policy/NAT/routing model; the differences are in coverage, granularity, and output assurance.

Dimension	fwforge	FortiConverter
Source coverage	Palo Alto (XML, set, Panorama, multi-vsyst)	Palo Alto plus 15+ other vendors — broader
App-ID granularity	Per-application signatures from the target's own FortiGuard DB (live, refreshable) + category fallback	Per-application signatures from a bundled licensed table
Security profiles	App-ID, URL (categories + URL lists), file, AV, WildFire→FortiSandbox, IPS — schema-certified; DLP pending	Broad profile coverage incl. DLP
IPS conversion	Severity + exact CVE crosswalk + FortiGuard-recommended baseline; posture-parity, stated plainly	No PA Threat-ID → FortiGuard signature crosswalk exists for either tool
Output validation	Schema-certified against the exact target firmware build	Not certified against the live target schema
Determinism	Byte-deterministic, git-diffable	Not deterministic
Provenance / findings	Per-line source provenance; md/JSON/HTML report; non-zero exit on error	Warnings as comments in config-all.txt
Version-delta scan	Yes — bidirectional feature / rename / default-flip	No
Object hygiene	Prunes, merges, dedups (acts on output)	Reports; discard is opt-in
Routing-aware dstintf	Longest-prefix-match from source routes	Falls back to any when missing (per docs)
Deployment	Single local package, fully offline	Heavier install; FGT→FGT path runs in cloud
FortiManager output	JSON-RPC import bundle (objects + package)	Yes — mature
Support model	Self-hosted / community	Official Fortinet support + service SLA

Where FortiConverter leads: vendor breadth, Data Filtering → DLP today, official support, and product maturity. **Where fwforge leads:** output assurance (schema certification against the exact build),

transparency, determinism, per-line provenance, version-delta awareness, active hygiene, fully-local operation, and App-ID / profile mappings sourced live from your own FortiGate rather than a bundled table.

8. Example output

An emitted, schema-certified excerpt showing per-application App-ID signatures, the IPS CVE crosswalk, and a policy carrying the full UTM stack:

```
config application list
edit "pan-appctrl-1"
set other-application-action block
config entries
edit 1
set application 15832 15817 37065 # Facebook, Gmail, Zoom
set action pass
next
end
next
end
config ips sensor
edit "ips-vuln-strict-spy-strict"
config entries
edit 1
set severity critical high
set action reset
next
edit 2
set cve CVE-2021-44228 # Log4Shell – exact cross-vendor match
set action reset
next
end
next
end
config firewall policy
edit 1
set name "trust-to-untrust"
set utm-status enable
set ssl-ssh-profile "certificate-inspection"
set application-list "pan-appctrl-1"
set ips-sensor "ips-vuln-strict-spy-strict"
set av-profile "av-av1-wf" # av-scan + FortiSandbox
set webfilter-profile "wf-url-strict"
next
end
```

Appendix: verification methodology

To keep category and type mappings honest, fwforge's PAN-OS conversion data was verified against a live FortiGate rather than from documentation alone:

- **The FortiGuard application-signature DB** (~3,300 signatures with their IDs) was read from the device at `/api/v2/cmd/db/application/name` — this drives per-application App-ID.
- **FortiGuard web categories** (the ~93 IDs used by URL-filtering) were read from `/api/v2/monitor/webfilter/fortiguard-categories`; file-filter file types from the `antivirus.filetype` data source.
- **IPS sensor filters, antivirus av-scan / fortisandbox enums, and webfilter urlfilter types** were taken from the device's CLI schema, so emitted severity / CVE / action tokens are valid for that exact build.
- **The emitted configuration** (App-ID, web-filter + urlfilter, file-filter, antivirus + FortiSandbox, IPS, interfaces and policies) was run through fwforge's own schema certifier against the device's CLI schema and returned 0 unknown tables and 0 unknown attributes.

Reference device: FortiGate-601F, FortiOS 8.0.0 build0167.